

ASSIGNMENT 20.5

AI Assisted Coding

2303A510Ao-Vishnu Vardhan

Batch 14

Task 1 – Password Storage Vulnerability Check

Task:

Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.

```
C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC > Q1Ha20.2.py > ...
1  #write a python program to Check signup script for insecure password storage
2  #Check whether passwords are stored in plain text.
3  #Improve the script using hashing (bcrypt or hashlib).
4
5  import bcrypt
6  def register_user(username, password):
7      # Hash the password using bcrypt
8      hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
9
10     # Store the username and hashed password (for demonstration, we will just print it)
11     print(f"Username: {username}")
12     print(f"Hashed Password: {hashed_password.decode('utf-8')}")
13 # Example usage
14 register_user("user1", "securepassword123")
```

OUTPUT :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/Q1Ha20.2.py
Username: user1
Hashed Password: $2b$12$4gLj3CYz5ZukBR3xIz06z0kID2N13Q9PkRwTsTeo1.9y6bcQeoqz
PS C:\Users\ADMIN>
```

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

```

C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> Q2ha20.5.py > ...
1  #Ask AI to generate a script that connects to an external API.
2  #Identify if API keys/passwords are written directly in code.
3  #Refactor using environment variables or config files.
4  import os
5  import requests
6  def get_api_data():
7      # Load API key from environment variable
8      api_key = os.getenv('API_KEY')
9      if not api_key:
10         raise ValueError("API_KEY environment variable not set")
11     url = "https://api.example.com/data"
12     headers = {"Authorization": f"Bearer {api_key}"}
13     response = requests.get(url, headers=headers)
14     if response.status_code == 200:
15         return response.json()
16     else:
17         raise Exception(f"API request failed with status code {response.status_code}")
18 # Example usage
19 if __name__ == "__main__":
20     try:
21         data = get_api_data()
22         print(data)
23     except Exception as e:
24         print(e)

```

OUTPUT:

```

PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe"
0.5.py
API_KEY environment variable not set

```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```

C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> Q3ha20.5.py > ...
1  #* Prompt AI to generate a Flask file upload example.
2  #* Check for vulnerabilities like uploading executable files.
3  #* Add validation for file type, size, and secure filename handling.
4  from flask import Flask, request, redirect, url_for, flash
5  from werkzeug.utils import secure_filename
6  import os
7  app = Flask(__name__)
8  app.secret_key = "secretkey"
9  # Configure upload folder
10 UPLOAD_FOLDER = 'uploads'
11 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
12 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
13 # Limit file size (2MB)
14 app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024
15 # Allowed extensions
16 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'pdf', 'txt'}
17 # Allowed MIME types
18 ALLOWED_MIME_TYPES = {
19     "image/png",
20     "image/jpeg",
21     "image/gif",

```

OUTPUT :



Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```
C > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > 4ha20.5.py > _
1 #Ask AI to generate code using os.system() or subprocess.
2 #Identify command injection risks with user input.
3 #Fix using safe subprocess calls and input validation.
4 import subprocess
5 def execute_command(user_input):
6     # Validate user input (allow only alphanumeric and limited special characters)
7     if not user_input.isalnum():
8         return "❌ Invalid input: Only alphanumeric characters are allowed."
9
10    # Use subprocess to execute the command safely
11    try:
12        result = subprocess.run(['echo', user_input], check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
13        return f"✅ Command output: {result.stdout.strip()}"
14    except subprocess.CalledProcessError as e:
15        return f"❌ Command execution failed: {e.stderr.strip()}"
16 # Example usage
17 if __name__ == "__main__":
18     user_input = input("Enter a command argument (alphanumeric only): ")
19     print(execute_command(user_input))
```

OUTPUT :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/4ha20.5.py
Enter a command argument (alphanumeric only):
```

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

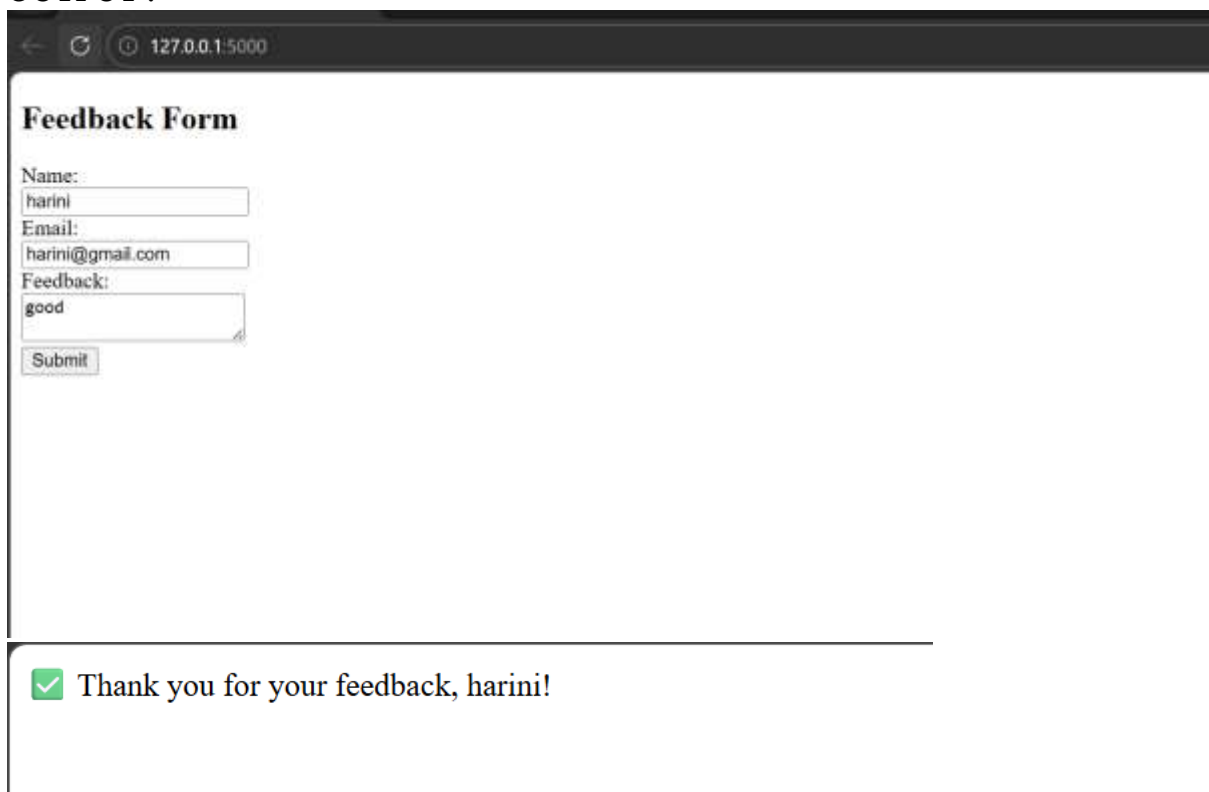
- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

A secure form resistant to injection attacks.

```
C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> Q5ha20.5.py ...
1  #* Ask AI to generate an HTML + Flask feedback form.
2  #* Identify missing validation and unsafe rendering.
3  #* Fix using server-side validation and sanitization.
4  from flask import Flask, request, render_template_string
5  import re
6  app = Flask(__name__)
7  # Simple HTML form template
8  form_template = '''
9  <!doctype html>
10 <title>Feedback Form</title>
11 <h2>Feedback Form</h2>
12 <form method=post>
13     <label for="name">Name:</label><br>
14     <input type=text name=name required><br>
15     <label for="email">Email:</label><br>
16     <input type=email name=email required><br>
17     <label for="feedback">Feedback:</label><br>
18     <textarea name=feedback required></textarea><br>
19     <input type=submit value=Submit>
20 </form>
21 '''
22 def sanitize_input(user_input):
```

OUTPUT:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page title is 'Feedback Form'. The form contains the following fields and values:

- Name: harini
- Email: harini@gmail.com
- Feedback: good

A 'Submit' button is located below the feedback text area. Below the form, a confirmation message is displayed: 'Thank you for your feedback, harini!' with a green checkmark icon.